

Relazione Progetto JBomberMan

Studentessa: Scarselli Ilaria

Matricola: 1918975

Corso: Metodologie Di Programmazione 2022-2023, presenza canale M-Z

Decisioni Progettuali e Realizzative

Adozione di Java Swing per interfaccia grafica

Nel corso dello sviluppo del progetto, la prima scelta importante è stata sicuramente quella riguardante la tecnologia da utilizzare per la creazione dell'interfaccia utente. Il mondo dello sviluppo Java offre diverse opzioni per costruire interfacce grafiche, si è optato per l'utilizzo di Java Swing invece di JavaFX. Questa decisione, dati gli obiettivi e le esigenze specifiche del progetto, è stata dettata da una maggiore immediatezza e la facilità d'uso della libreria Swing, che si presta per progetti di dimensioni non eccessive, come JBomberMan, per la realizzazione di semplici interfacce senza risultare troppo tediosa nell'uso, seppur dovendo disegnare fisicamente le varie componenti sullo schermo.

Design Pattern utilizzati

Il primo Design Pattern che è possibile riconoscere è MVC, un design pattern architetturale che prevede la divisione del progetto in tre moduli distinti: controller, model e view. In particolare, questi ultimi non comunicano direttamente tra loro, ma esclusivamente tramite il controller, che svolge il ruolo di intermediario. In questo modo si aumenta la riusabilità del codice rendendo il model completamente indipendente dalla view e riutilizzabile per altri progetti simili. E' dunque il controller, nel progetto, che invia i valori secondo cui la view deve aggiornare la vista e controlla e gestisce al tempo stesso gli aggiornamenti del modello.

Il secondo pattern utilizzato è stato Observer-Observable, pattern comportamentale. Nel caso di questo progetto, la classe Entity estende Observable, ma l'Observable effettivo è il PlayerModel. Questo perché Entity è la radice di una gerarchia di classi di cui PlayerModel fa parte, ma PlayerModel può estendere solo

una classe. Il PlayerModel notifica le sue modifiche man mano che si aggiorna al GamePanel, la classe che implementa l'interfaccia Observable, mediante i metodi standStill() e movePlayer().

Inoltre ho implementato il pattern Singleton (un pattern creazionale che permette l'esistenza di una singola istanza della classe che lo implementa) in Classi quali ModelAssets, PlayerModel e GamePanel.

Utilizzo di stream

Gli stream sono stati utilizzati in particolare per gestire le bombe durante la partita, per creare una List di indici per gestire le bombe in fase di detonazione e per gestire la lista delle bombe lato view per disegnarle e aggiornarle mediante cicli forEach.

Riproduzione audio

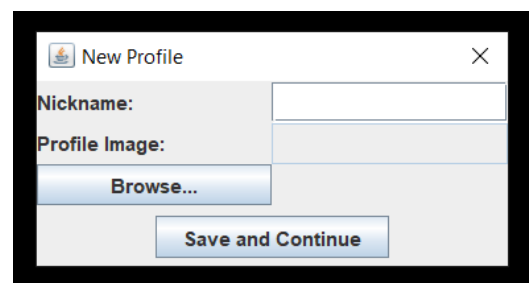
Per riprodurre file audio non è stato usato l'AudioManager fornito, ma ho preferito l'utilizzo di una classe Sound costruita appositamente che fa uso di AudioInputStream e prende in input file .wav.

Animazioni

Le animazioni dei mostri, del personaggio, delle bombe e delle esplosioni sono state realizzate mediante l'utilizzo di contatori appositi e flag per permettere di gestire le tempistiche del cambio di sprite da disegnare. I contatori e i flag sono stati gestiti nella parte del controller (principalmente dalla classe JBomberMan), mentre il GamePanel nella view si è occupato della parte di disegno effettivo e del caricamento degli sprites. Le varie schermate sono state gestite dalla classe UI nella view (che le disegnava a ogni update, controllata dal GamePanel) e dalle classi GameStateManager e KeyHandler nel controller.

Gestione dell'utente

Nel menù principale è possibile scegliere tra una nuova partita e caricare un profilo utente già creato. Nel caso si scelga una nuova partita sono presenti un semplice JDialog per permettere la scelta di un nickname e di una immagine da utilizzare come immagine profilo:



Nel caso si scelga Load, verranno caricati i dati dell'utente.

I punti accumulati durante un livello verranno trasformati in exp solo al superamento del livello corrente (ovvero quando, una volta sconfitti tutti i mostri, si raggiunga la porta del livello) e vengono invece persi in caso di abbandono o di sconfitta del giocatore. Vengono considerate partite vinte quelle in cui vengono superati entrambi i livelli, mentre non vengono considerate come perse quelle abbandonate. E' possibile navigare nei menù di gioco con la tastiera, solo per la creazione del nuovo profilo può essere utilizzato il mouse.



Nemici

Ci sono due tipi di nemici, con comportamento e numero di vite diverse: i pipistrelli, con movimento randomico e più imprevedibile, più veloci, ma con una sola vita; gli slime, più lenti e che hanno la possibilità di muoversi solo in orizzontale o in verticale in base al parametro che gli è stato assegnato, ma con più vita.

Power-Up

Ci sono tre tipi di power-up:

- Aumento di velocità del giocatore;
- Aumento del numero di bombe che è possibile piazzare;
- Aumento della potenza delle bombe intesa come la distanza a cui possono arrivare seguendo un pattern a croce.

L'esplosione distrugge le soft-tiles, ma viene invece bloccata dai muri o dalle hard-tiles.

Vite del giocatore

Il giocatore dispone di due tipi di vite: i cuori e i bomberMan. Ha in tutto 4 bomberMan e ognuno è composto da 3 cuori. Tipicamente i danni ricevuti da un mostro sottraggono un cuore, mentre farsi sorprendere da un'esplosione costa al giocatore ben 3 cuori. Quando si perde unBomberMan, la posizione viene riportata a quella di default per il livello e si perdono i power-up acquisiti.

References

Per la realizzazione del progetto sono stati consultati vari tutorial e fonti ufficiali, tra cui:

- [How to Write Doc Comments for the Javadoc Tool | Oracle Italia](#)
- [javadoc \(oracle.com\)](#)
- [GB Factory Code - YouTube](#)
- [RyiSnow - YouTube](#)

Sono state utilizzare risorse da varie fonti per sprite e audio, tra cui:

- [SNES - Super Bomberman - The Spriters Resource \(spriters-resource.com\)](#)
- [RyiSnow - YouTube](#); [Blue Boy Adventure - Google Drive](#)